

PRÁCTICA_ 11

Implementa una app que muestra información de películas actuales.

1. Genera un proyecto nuevo de tipo tabs
2. Cambia los iconos y el texto de las tres pestañas



Los iconos son: home, search, start-outline

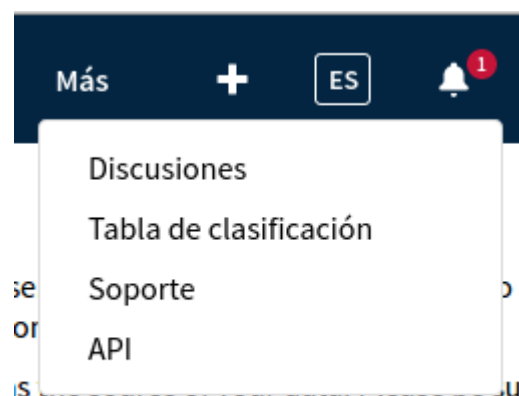
3. Crea un API Key en TheMovieDB

Nos registramos en la página TheMovieDB.org, este será el API que usaremos para obtener en tiempo real la información de las películas que tienen almacenadas en su bbdd.

Una vez registrados, tenemos que generar el api key, en el icono de mi imagen de usuario abro las opciones de configuración. Edito mi perfil y busco Solicitar Clave de API.

Tendremos que rellenar un formulario con datos sobre nuestra app y nuestros.

Una vez que tenemos nuestro api key, veamos cómo funciona esta web



En la opción API, Descubrir ejemplos tenemos una serie de ejemplos de url válidas para obtener distinta información que nos ofrece este api.

4. Vamos a utilizar el primer ejemplo, películas estrenadas entre dos fechas. Nada más arrancar la app mostraremos las publicadas en una fecha reciente. Antes de codificar, probamos con postman que este servicio funciona.

A la url ejemplo le tenemos que añadir `https://api.themoviedb.org/3`

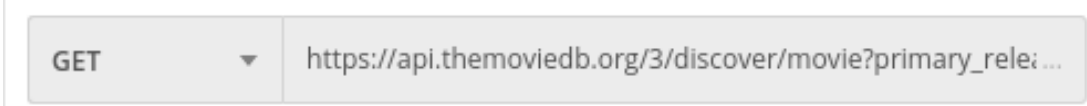
Por tanto, nos quedaría

`https://api.themoviedb.org/3/discover/movie?primary_release_date.gte=2014-09-15&primary_release_date.lte=2014-10-22`

Si lanzamos esta petición, no obtendremos las películas sino un error porque no nos hemos identificado en la petición lanzada. A esa url le añadimos

`https://api.themoviedb.org/3/discover/movie?primary_release_date.gte=2014-09-15&primary_release_date.lte=2014-10-22&api_key=valor`

En postman podemos cambiar los valores de los parámetros, por ejemplo, las fechas para ver películas recientes



The screenshot shows the Postman interface for a GET request. The URL bar contains the URL with truncated parameters. Below the URL bar, the 'Params' tab is selected, displaying a table of query parameters.

	KEY	VALUE	DES
☒	primary_release_date.gte	2021-01-01	
☒	primary_release_date.lte	2021-03-30	
☒	api_key	fecbd5c4d8a1afa1676456082235f...	
	Key	Value	Des

La información nos aparece en inglés, podemos añadir un nuevo parámetro para verla en español porque este api así lo permite. Añado directamente el parámetro language en la casilla debajo de api_key, el valor es y se añadirá automáticamente a la url

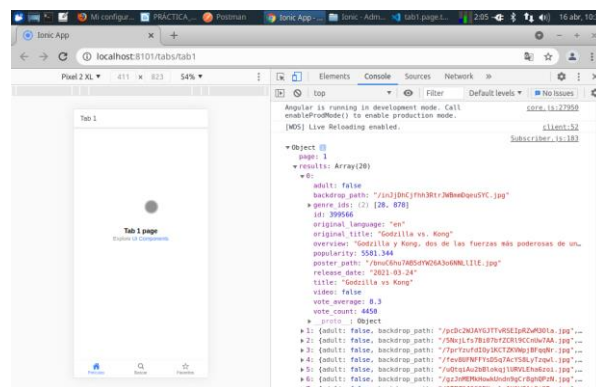
Este api también nos permite que las imágenes salgan con el texto en español, para ello añadimos el parámetro include_image_language con valor es

PARTE I: TAB PELÍCULAS

5. Creamos el servicio en nuestra aplicación para obtener la información, no olvidemos importar el módulo HttpClient en el appmodule

Implementamos un método en el servicio getMovies() para obtener la información de la url probada en postman

Para probar que traemos bien la información, en tab1 lo inyectamos y probamos a lanzar el servicio y escribir en consola



6. Crea las correspondientes interfaces para definir los tipos de información que manejaremos

Recuerda que debes copiar la respuesta obtenida en postman y generar con la extensión JSON To TS las interfaces.

Las exportamos y cambiamos de nombre: IMovies, IMovie

Añadimos la parametrización en el servicio, un array de IMovie en la clase tab1 para almacenar los resultados que nos retorna el servicio.

```
ngOnInit(){
  this.dataMov.getMovies().subscribe(
    (resultado : IMovies) => {
      this.ListaPeliculas=resultado.results;
    }
  );
}
```

No hay cambios en cuanto al funcionamiento, sólo hemos definido los tipos de los datos con los que trabajaremos.

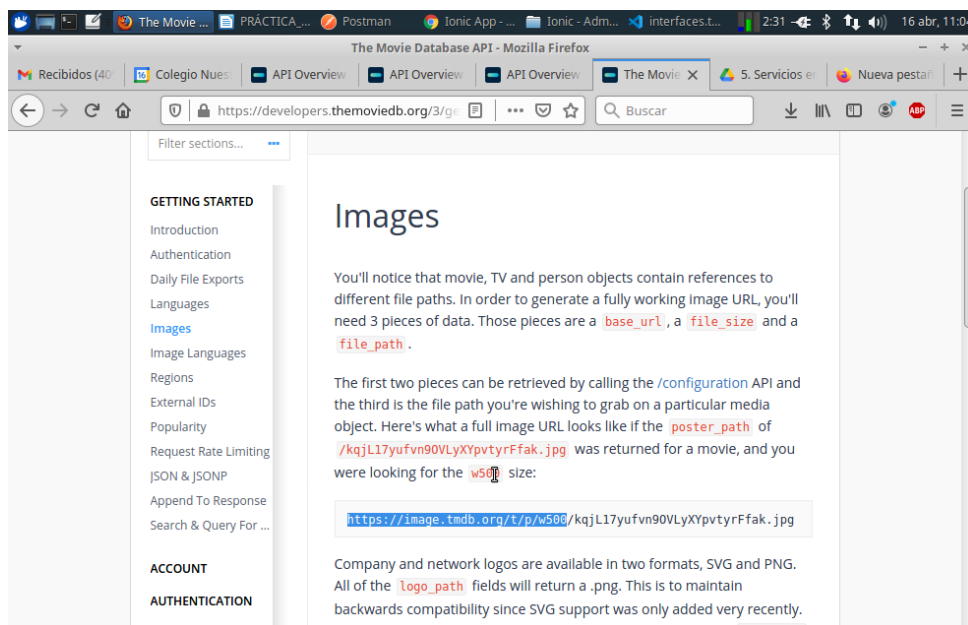
7. Mostramos las imágenes de las películas

En el template de tab1 borramos todo el html y añadimos un slide para ver las películas

En la documentación de la api podemos obtener las url que tendremos que anteponer a las variables `backdrop_path` y `poster_path` de nuestra interfaz. En esas variables el api nos proporciona la parte diferente de la url de cada imagen asociada a cada película, hay que concatenarle una parte que es común a todas las imágenes

<https://image.tmdb.org/t/p/w500>

donde w500 es el tamaño de la imagen



Ambas propiedades nos darán una imagen de la película, un tipo póster y otra tipo carátula. Usaremos backdrop (póster)

Para no estar copiando y concatenando siempre estos valores, vamos a generar unos pipes que nos automaticen esta tarea.

Creamos un módulo y lo llamamos pipes

Creamos un pipe y lo llamamos imagen: ionic g pipe pipes/imagen

```
@Pipe({
  name: 'imagen'
})
export class ImagenPipe implements PipeTransform {

  transform(value: unknown, ...args: unknown[]): unknown {
    return null;
  }

}
```

En el módulo exporto el ImagenPipe

Codificamos el pipe

```
const URL="https://image.tmdb.org/t/p/";
@Pipe({
  name: 'imagen'
})
export class ImagenPipe implements PipeTransform {

  transform(img: string, size:string='w500'): string {
    if (!img)
      return null;
    const imgURL=`${ URL }/${ size }/${ img }`;
    //const imgURL= URL + size + img;
    return imgURL;
  }

}
```

Lo usamos desde el template

```
<ion-content [fullscreen]="true">
  <ion-slides [options]="slideOpts">
    <ion-slide *ngFor="let p of listaPeliculas">
      <ion-card>
        <img [src]="p.backdrop_path | imagen">
      </ion-card>
    </ion-slide>
  </ion-slides>
</ion-content>
```

Donde slideOpts es un array declarado en la clase con las propiedades

```
slideOpts={
  slidesPerView :1.3,
  freeMode:true
}
```

Podemos tener almacenada una imagen por si alguna película no viniera con póster, para cargarla en ese caso. Implementa tú este código.

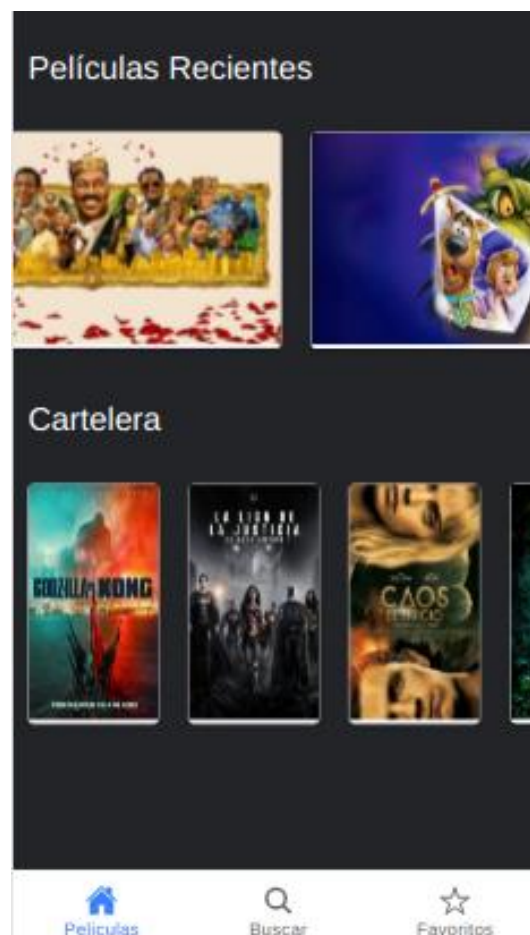
8. Optimizamos el almacenamiento de las url y el api key utilizando los archivos environment. Crearemos tres variables para guardar:

- apikey
- URL base del servicio: <https://api.themoviedb.org/3>
- URL base de las imágenes: <https://image.tmdb.org/t/p/>

Tanto en el pipe como en el servicio utilizamos las variables del environment

9. Mejoramos nuestro código incorporando los componentes adecuados. Puesto que necesitaremos mostrar películas en las tres tabs, si implementamos el código en un componente podremos reutilizarlo en todas, en lugar de repetirlo en cada uno de los templates.

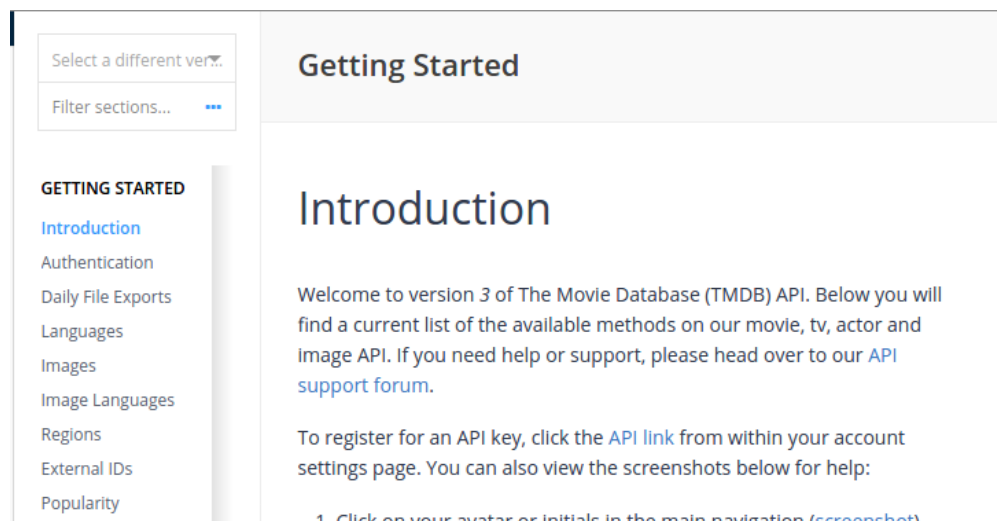
- Genera un módulo llamado components
- Genera un componente dentro de ese módulo: slidebackdrop. No olvidar exportarlo y declararlo en el módulo creado. En este módulo también hay que importar IonicModule, pipemodule
- Declara una variable en el componente para recibir el array de películas desde su padre (@Input) y declara también aquí el array slideOpts
- Genera otro componente para mostrar las imágenes tipo póster (repite el proceso análogo al backdrop). Añade una clase css para que las imágenes tengan un ancho de 110px y un alto de 160px. En las opciones del slide, cambia a 3.3 para que se vean más carteles.



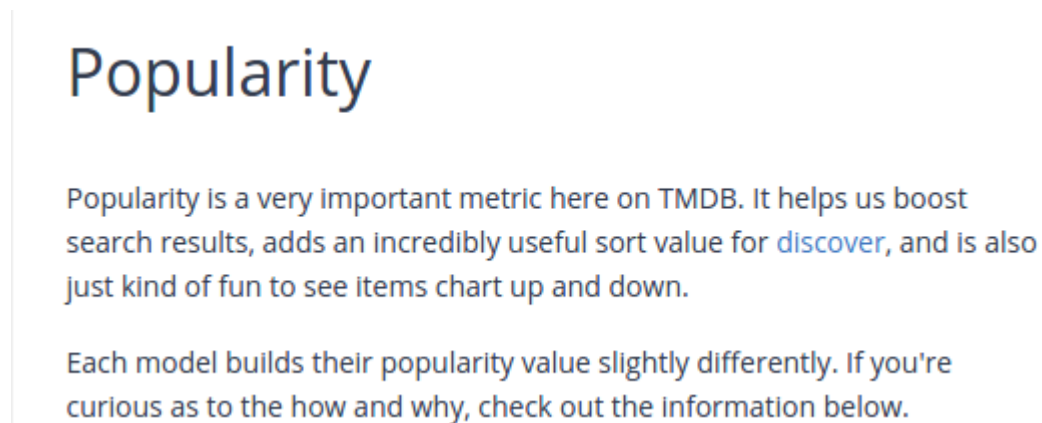
El resultado hasta el momento debe ser algo así

10. Mostremos las películas más populares, para ello implementaremos otro método en nuestro servicio: `getPopulares()`

En la documentación de la api, en la sección de popularity, encontraremos la forma de pedir las películas más populares



Seleccionamos la opción popularity y veremos la documentación sobre como trabajar esta sección,



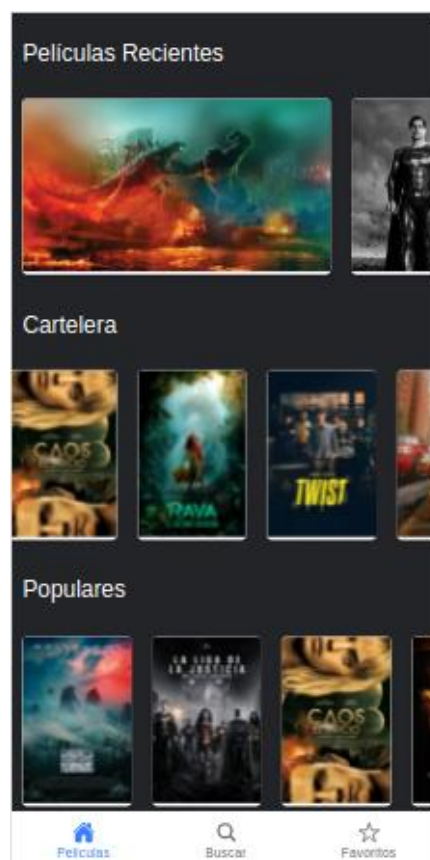
Pulsamos sobre el enlace y nos llevará a otra página dónde nos explican el uso de los posibles parámetros de la URL, incluido el ordenar por popularidad las películas

Query String

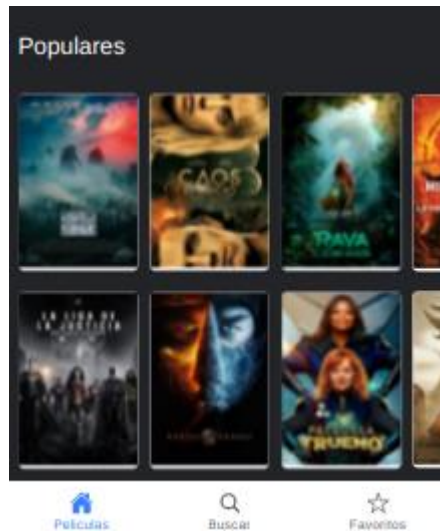
api_key	string	default: <<api_key>>	required
language	string	Specify a language to query translatable fields with. minLength: 2 pattern: ([a-z]{2})-([A-Z]{2}) default: en-US	optional
region	string	Specify a ISO 3166-1 code to filter release dates. Must be uppercase. pattern: ^[A-Z]{2}\$	optional
sort_by	string	Choose from one of the many available sort options. Allowed Values: , popularity.asc, popularity.desc, release_date.asc, release_date.desc.	optional

Por tanto tendré que añadir en mi URL el parámetro `sort_by=popularity.desc` (utilizamos este valor pero podríamos coger el que quisiéramos) y eliminar los parámetros de las fechas.

El resultado será:



11. Vamos a cambiar el aspecto con el que mostramos las películas populares. Vamos a mostrarlas en dos filas, ambas en un slide diferente



Evidentemente si cargo el array dos veces, nos aparecen en dos filas pero repetida arriba y abajo la misma película.

Creamos un pipe (pares) para determinar cómo mostrar el array de películas populares en dos. El objetivo es recibir un array y transformarlo en otro que a su vez contenga arrays dentro, uno para las películas de arriba y otro para las de abajo. Implementa la función en código javascript que, dado el array de películas populares, genera otro array de manera que cada casilla es a su vez otro array con dos películas, la de la primera fila y la de la segunda. Este array tendrá la mitad de casillas que el array fuente.

12. En este apartado vamos a utilizar @Output, es decir vamos a emitir desde el componente slidepares una petición a su “padre” tab1 para que cargue las películas “de forma paginada” en lugar de todas a la vez. Colocaremos un botón y mostraremos sólo las películas que nos caben en la pantalla, al pulsar el botón le diremos al servicio que nos de la siguiente “tanda” y así sucesivamente hasta que no haya más en el array.

a. Probemos con postman a paginar los resultados que nos retorna una URL

Añado en los parámetros uno nuevo llamado page y le indico el n.º de página de los resultados que quiero ver.

GET

https://api.themoviedb.org/3/discover/movie?sort_by=popu

Params

Auth

Headers (6)

Body

Pre-req.

Tests

Settings

KEY	VALUE
☒ sort_by	popularity.desc
☒ api_key	fecbd5c4d8a1afa1676456082235f...
☒ language	es
☒ page	2

b. Añadimos el código para que el componente slidepares emita una llamada a su padre, pidiendo la siguiente tanda de películas

```
@Input() listaPelículas : IMovie[]=[];  
@Output() cargarMas =new EventEmitter();
```

EventEmitter tiene que ser importado de @angular/core

c. Colocamos un botón para ejecutar esa llamada y cargar más películas

```
<ion-slides [options]="slideOpts" >  
  <ion-slide *ngFor="let p of listaPelículas | pares " >  
    <ion-row >  
      <ion-col size="12" *ngFor="let item of p">  
        <ion-card >  
          <img class="poster" [src]="item.poster_path | imagen">  
        </ion-card>  
      </ion-col>  
    </ion-row>  
  </ion-slide>  
  
  <ion-slide class="slidemas">  
    <ion-button (click)="onClick()" expand="full" size="large" class="mas">  
      <ion-icon slot="icon-only" name="add"></ion-icon>  
    </ion-button>  
  </ion-slide>  
</ion-slides>
```

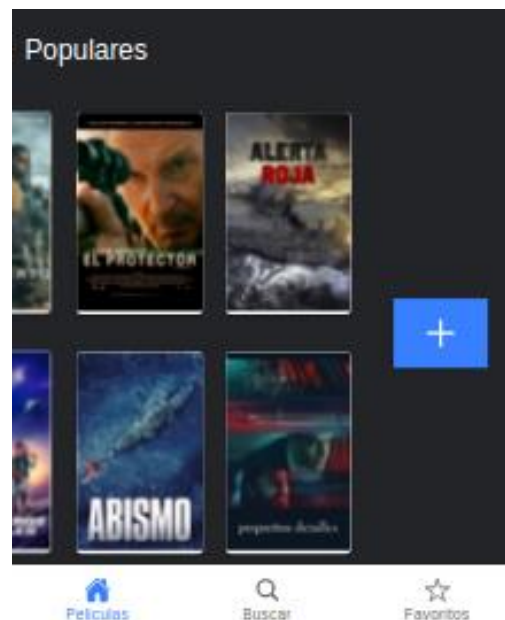
d. Añadimos otro slide para contener al botón.

e. Añadimos un botón con esas propiedades

f. Definimos dos clases css para que este slide tenga el mismo alto que los otros (podemos inspeccionar en html sus propiedades para ver cuanto mide el height). En mi caso

```
.mas{
  top:13%;
}
.slidemas{
  height: 386px;
}
```

El aspecto será de momento



g. Una vez “pintado” el botón vamos a añadirle la funcionalidad, en su evento click tendremos que programar en la clase que este componente, cuando le pulsan el botón, emita una petición a su padre(en este caso tab1) para que le envíe más películas

```
export class SlideparesComponent implements OnInit {  
  
  @Input() listaPelículas : IMovie[]=[];  
  @Output() cargarMas =new EventEmitter();  
  slideOpts={  
    slidesPerView :3.3,  
    freeMode:true,  
    spaceBetween:-10  
  }  
  constructor() {  
  
  }  
  
  ngOnInit() {}  
  
  onClick(){  
    this.cargarMas.emit();  
  }  
}
```

En tab1, tenemos que “escuchar” este evento que nos emite nuestro componente slidepares. Para ello en el template

```
<app-slidepares [listaPelículas]="listaPelículasPopulares"  
  (cargarMas)="cargarMas()"> </app-slidepares>  
</ion-content>
```

cargarMas() será un método de la clase que conectará con el api de películas para traer más películas.

En el servicio tenemos que cambiar el método getPopulares para que reciba por parámetro la página de resultados que queremos obtener:

declaro una variable para ir contando el número de página por el que voy

```
getPopularity(){  
  this.pagePopularity++;  
  const query=URL+'/discover/movie?sort_by=popularity.desc&page='+this.pagePopularity+'&'  
  | +apikey+'&language=es';  
  return this.http.get<IMovies>(query);  
}
```

En tab1, el método cargarMas tiene el mismo código que implementamos en el ngOnInit

```
this.dataMov.getPopularity().subscribe(  
  

```

```
(resultado : IMovies)=>{  
  

```

```
this.listaPelículasPopulares=resultado.results;}
```

Si probamos este código veremos que al pulsar el botón nos carga otra tanda de películas, pero machaca las anteriores. Para evitar esto, el código del método cargarMas debe ser

```
cargarMas(){
  this.dataMov.getPopularity().subscribe(
    (resultado : IMovies)=>{
      const l=[...this.listaPeliculasPopulares,...resultado.results];
      this.listaPeliculasPopulares=l;
    }
  );
}
```

la variable l será el array dónde voy concatenando las películas que tenía de antes, con el nuevo resultado que me manda el array.

Prueba la ejecución y verás como las nuevas películas se van añadiendo al final sucesivamente.

13. Lo siguiente que haremos será implementar el código necesario para mostrar los detalles de una película cuando nos “clikan” en la imagen. Mostraremos otra ventana modal

a. Genero un componente llamado detalle

b. Vamos a empezar por el componente slidebackdrop, en los otros dos será análogo

La ventana queremos que sea modal, para ello inyectamos el ModalController de ionic en el constructor de slidebackdrop

c. En el template del componente, implementamos el evento click para el ion-card, pasamos como parámetro el id de la película

```
<ion-slides [options]="slideOpts">
  <ion-slide *ngFor="let p of listaPeliculas">
    <ion-card (click)="verDetalle(p.id)">
      <img [src]="p.backdrop_path | imagen">
    </ion-card>
  </ion-slide>
</ion-slides>
```

d. Implementamos el método verDetalle() en la clase. Recuerda el uso del componente ion-modal

```
async verDetalle(id:string){  
    const ventanaModal= await this.mod.create(  
        {  
            component:DetalleComponent,  
            componentProps: { id}  
        }  
    );  
    ventanaModal.present();  
}
```

Declaramos un `@Input ()` en la clase detalle para recibir ese id

e. Veamos la documentación del api para saber cómo obtener la información de cada película, sus actores....

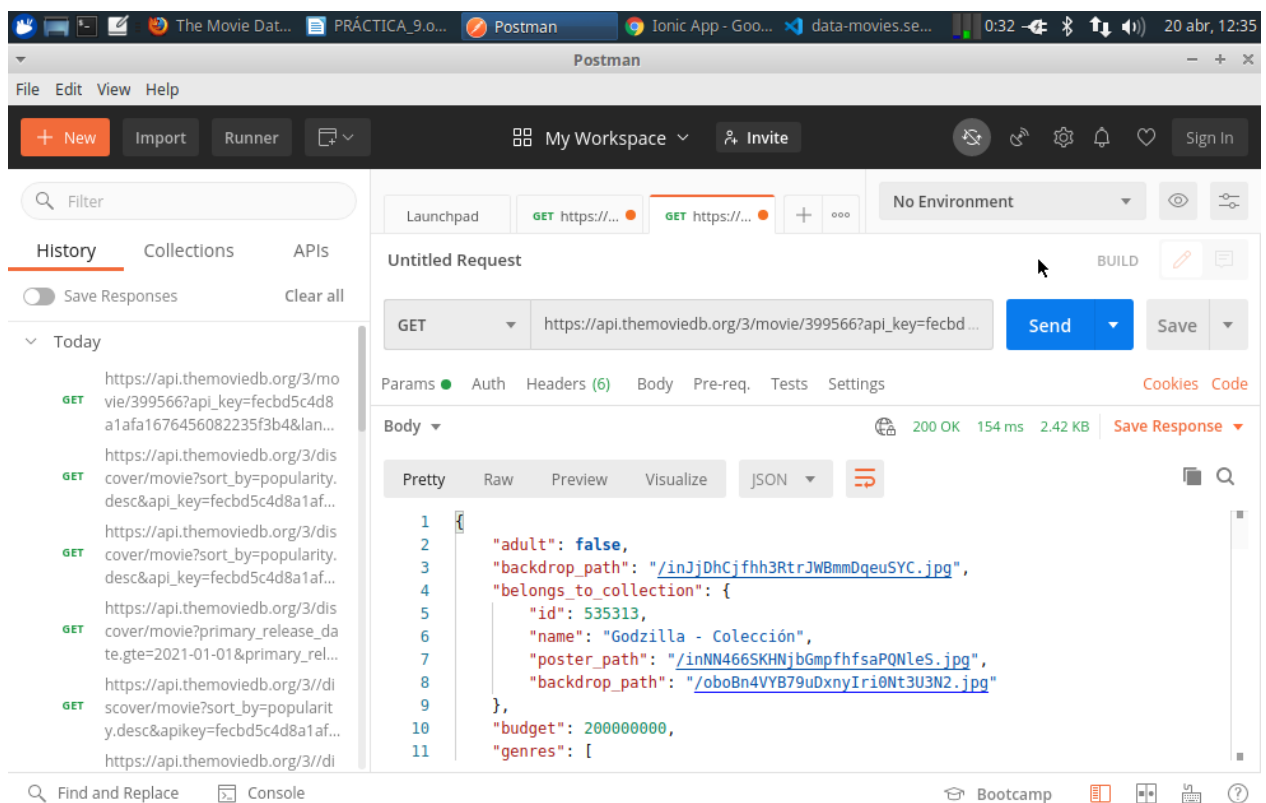
MOVIES	
GET	Get Details
GET	Get Account...
GET	Get Alternat...
GET	Get Changes
GET	Get Credits
GET	Get Externa...
GET	Get Images
GET	Get Keywords
GET	Get Lists

La opción en azul no dará los detalles de la película, Get Credits nos dará la información sobre los actores

Construimos la URL para ver los detalles de la película, usamos postman para probar la petición y generar las interfaces apropiadas a lo que nos retorna. Como no sé qué id tienen las películas, escribo uno en consola para generar la url

```
export class DetalleComponent implements OnInit {  
  
  @Input() id:string;  
  constructor() { }  
  
  ngOnInit() {  
    console.log(this.id);  
  }  
}
```

Generamos la url y la probamos en postman



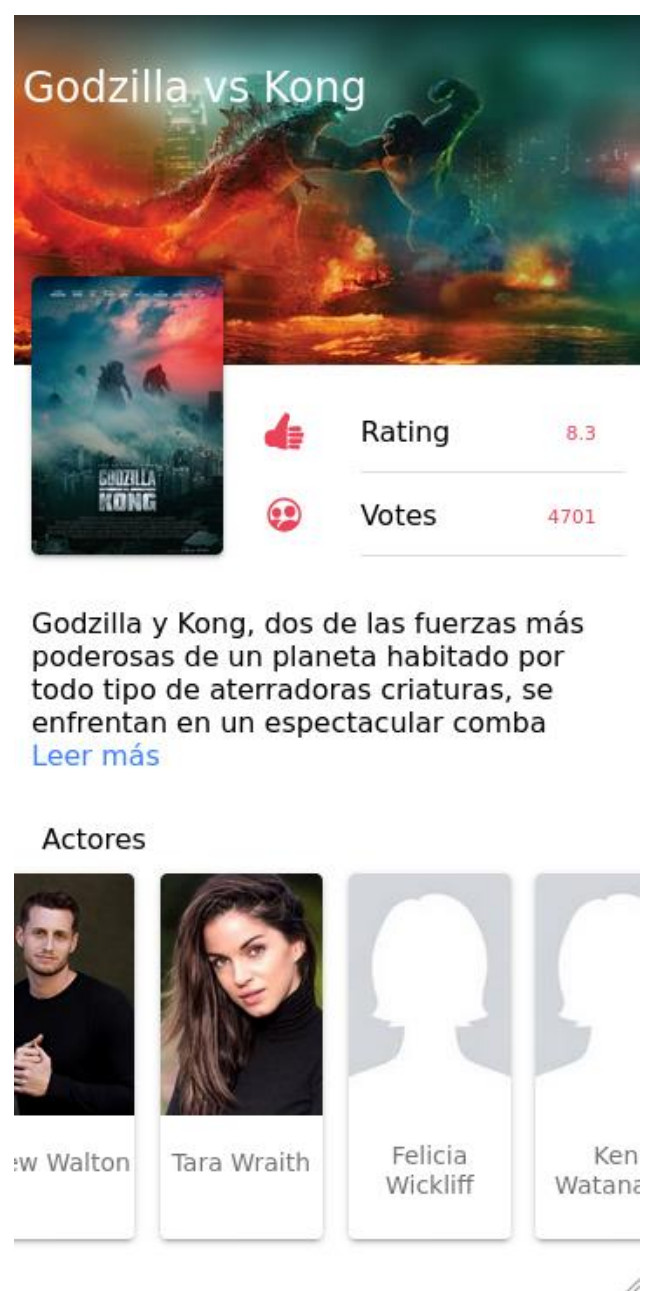
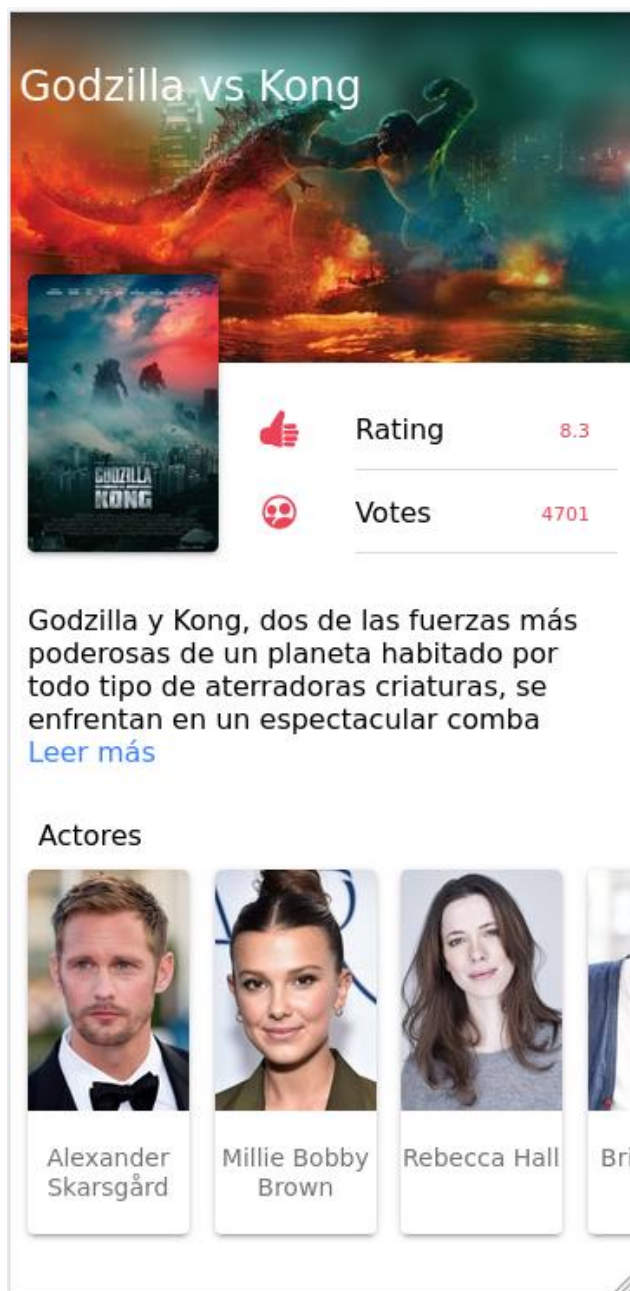
Ya podemos crear las interfaces

f. Añadimos el correspondiente método al servicio

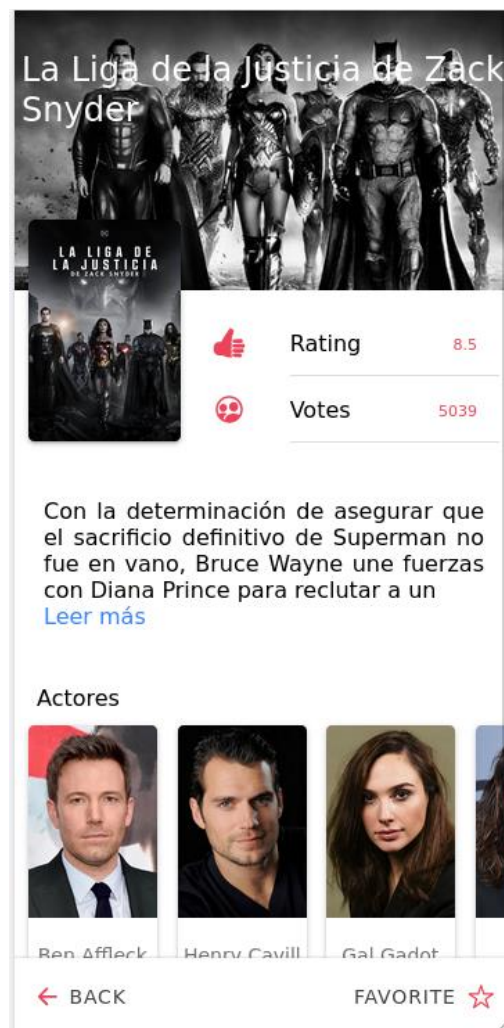
g. Repetimos el proceso para los actores (get credits)

h. Mostremos esos detalles

Se te proporciona un archivo con los estilos que yo he usado, el template y la imagen no-avatar. Puedes diseñar tú la página detalles a tu gusto o copiar el diseño que se te muestra en la imagen. Debes entender el código de los archivos proporcionados.



14. Añade un footer para poner dos opciones: regresar a la página anterior y añadir a favoritos



Para implementar la opción Back, sólo es necesario cerrar la ventana modal.

La opción Favoritos la implementaremos más tarde

PARTE II: TAB BUSCAR

15. Diseñamos una página para buscar películas

a. En el template del tab2 incluimos un ion-search

b. Implementamos el servicio para buscar la película solicitada. Para ello tenemos que ver como trabaja el api este tipo de peticiones. Como siempre entramos en la documentación y elegimos la opción SEARCH. Nos ofrecen varias opciones de búsqueda, nosotros utilizaremos search movies.

Estudia la información que te proporciona el api para saber que parámetros podrás usar en la búsqueda, cual es el resultado obtenido (Response),....

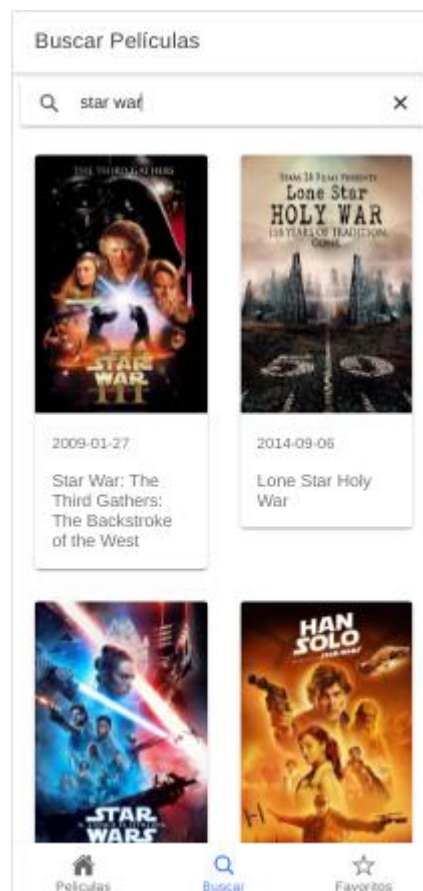
Utiliza postman para ver los resultados y comprobar si tienes definidas todas las interfaces que necesitarás.

c. Desde la clase de tab2, llamamos al método del servicio pasando como parámetro el texto que nos introducen en el ion-search

d. Diseñamos el template de tab2 para que se muestren en dos columnas tal y como se ve en la siguiente imagen



e. Mejoremos un poco el diseño. Puedes decidirlo tú o bien seguir los pasos necesarios para que te quede algo así:



f. Hagamos que al clikear una de las películas nos muestre su detalle.